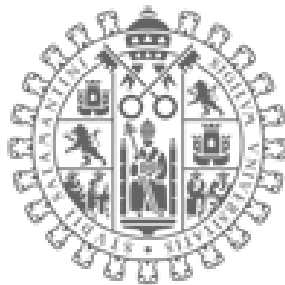


Calidad y Auditoría

Investigación y aplicación de herramientas para pruebas unitarias en un lenguaje de programación (Python)



**VNiVERSiDAD
D SALAMANCA**

CAMPUS OF INTERNATIONAL EXCELLENCE

Miguel Barranquero Diez

Índice

1. Introducción	4
2. Herramientas	5
2.1. Herramienta 1: Pytest	5
2.1.1. Características principales	5
2.1.2. Ventajas y desventajas	6
2.1.3. Compatibilidad con sistemas de integración continua (CI/CD)	6
2.1.4. Soporte para la automatización de pruebas	7
2.2. Herramienta 2: Unittest	8
2.2.1. Características principales	8
2.2.2. Ventajas y desventajas	8
2.2.3. Compatibilidad con sistemas de integración continua (CI/CD)	9
2.2.4. Soporte para la automatización de pruebas	9
2.3. Comparación Pytest vs Unittest	10
3. Configuración y Uso de pytest	11
3.1. Instalación de pytest	11
3.1.1. Requisitos previos	11
3.1.2. Proceso de instalación	11
3.2. Configuración del Entorno de Desarrollo	11
3.2.1. Estructura del proyecto	11
3.2.2. Archivo de configuración pytest.ini	12
3.2.3. Integración con entornos de desarrollo	13
3.3. Creación de Pruebas Unitarias	14
3.3.1. Estructura básica de un test	14
3.3.2. Fixtures: Reutilización de código de prueba	14
3.3.3. Aserciones: Verificación de resultados	14
3.3.4. Pruebas parametrizadas	15
3.3.5. Verificación de excepciones	15
3.4. Ejecución de Pruebas	17
3.4.1. Ejecución básica	17
3.4.2. Ejecución en modo verbose	17
3.4.3. Análisis de cobertura de código	18
3.4.4. Interpretación de resultados	18
4. Desarrollo Práctico de Pruebas Unitarias	20
4.1. Sistema Desarrollado: Calculadora	20
4.1.1. Descripción del componente	20
4.1.2. Implementación del código fuente	20
4.1.3. Características de implementación	21
4.2. Pruebas Implementadas	22
4.2.1. Código de las pruebas unitarias	22
4.2.2. Análisis de las pruebas implementadas	25
4.2.3. Justificación de casos de prueba	25
4.3. Resultados Obtenidos	26
4.3.1. Ejecución exitosa de las pruebas	26
4.3.2. Análisis de cobertura de código	26

4.3.3. Interpretación de calidad del sistema	27
4.3.4. Limitaciones y trabajo futuro	27
Bibliografía	28

1. Introducción

Las pruebas unitarias constituyen uno de los pilares fundamentales en el desarrollo de software moderno, permitiendo verificar el correcto funcionamiento de componentes individuales de forma aislada y automatizada [1]. En un contexto donde la calidad del software es crítica y los ciclos de desarrollo son cada vez más ágiles, contar con un conjunto robusto de pruebas unitarias se ha convertido en una práctica esencial para garantizar la fiabilidad, mantenibilidad y escalabilidad de las aplicaciones [2].

El presente informe documenta el proceso completo de investigación, configuración e implementación de pruebas unitarias para un sistema software desarrollado en **Python**. Este lenguaje ha sido seleccionado debido a su amplia adopción en la industria, su sintaxis clara y legible, y la robusta ecosistema de herramientas de testing disponibles [3]. Python, como lenguaje interpretado y de tipado dinámico, presenta características particulares que lo hacen especialmente interesante para el estudio de metodologías de pruebas automatizadas [4].

El trabajo se estructura en torno a tres objetivos principales. En primer lugar, se realizará un análisis comparativo exhaustivo de dos frameworks de testing ampliamente utilizados en el ecosistema Python: **pytest** y **unittest** [5]. Esta comparación abarcará aspectos técnicos, funcionales y prácticos, incluyendo características principales, ventajas y desventajas, compatibilidad con sistemas de integración continua (CI/CD) y capacidades de automatización [6].

En segundo lugar, se documentará de manera detallada el proceso completo de configuración de un entorno de pruebas funcional, desde la instalación de las herramientas necesarias hasta la integración con entornos de desarrollo modernos. Este proceso incluirá la estructura de proyecto recomendada, configuraciones específicas y mejores prácticas para la organización del código de pruebas [7].

Finalmente, se desarrollará e implementará un conjunto completo de pruebas unitarias sobre un sistema real: un gestor de biblioteca que simula operaciones típicas de préstamo, devolución y gestión de libros. Este sistema, aunque sencillo, incorpora suficiente complejidad para demostrar diferentes tipos de pruebas: validación de entradas, manejo de excepciones, pruebas de estado, pruebas parametrizadas y análisis de cobertura de código [8].

A través de este enfoque práctico y sistemático, el informe busca proporcionar una guía completa que permita comprender no solo los aspectos técnicos de las pruebas unitarias, sino también su valor estratégico en el ciclo de vida del desarrollo de software, demostrando cómo una suite de pruebas bien diseñada contribuye significativamente a la calidad y confiabilidad del producto final [9].

2. Herramientas

La elección de una herramienta adecuada para pruebas unitarias es fundamental para el éxito de un proyecto de software. En el ecosistema de Python, existen diversos frameworks de pruebas, cada uno con enfoques, características y aplicaciones específicas. Para este análisis, se seleccionaron dos de las opciones más representativas y utilizadas: pytest y unittest. Estos frameworks ilustran enfoques contrastantes:

- Pytest, como una solución moderna y orientada a la eficiencia.
- Unittest, como el framework estándar basado en el patrón xUnit.

A continuación, se presenta apartados detallados de cada uno, evaluando sus características, ventajas, limitaciones y capacidades de integración.

2.1. Herramienta 1: Pytest

Pytest es un framework de testing maduro y completo que se ha convertido en el estándar de facto para pruebas en Python. Lanzado inicialmente en 2004 y con desarrollo activo continuo, pytest se caracteriza por su filosofía de simplicidad y su capacidad para escalar desde pruebas básicas hasta suites de testing complejas en proyectos empresariales [10], [8].

2.1.1. Características principales

Pytest destaca por ofrecer un conjunto de características que lo diferencian significativamente de otros frameworks [8], [10]:

- **Sintaxis simple:** A diferencia de frameworks basados en xUnit, pytest no requiere que los tests hereden de clases especiales ni utilicen métodos con nombres específicos. Los tests son simplemente funciones que comienzan con `test_`, lo que reduce drásticamente el boilerplate code y hace el código más legible y mantenible.
- **Descubrimiento automático de tests:** pytest implementa un sistema inteligente de descubrimiento que automáticamente encuentra y ejecuta todos los tests en el proyecto siguiendo convenciones de nomenclatura predefinidas (archivos `test_*.py` o `*_test.py`, funciones y métodos que comienzan con `test_`), eliminando la necesidad de registrar manualmente los tests.
- **Sistema de fixtures avanzado:** Proporciona un mecanismo de fixtures extremadamente flexible que permite definir recursos, configuraciones y datos de prueba reutilizables. Las fixtures soportan diferentes alcances (function, class, module, session), pueden solicitar otras fixtures automáticamente mediante inyección de dependencias, y se limpian automáticamente tras su uso.
- **Parametrización nativa:** Incluye el decorador `@pytest.mark.parametrize` que permite ejecutar el mismo test con múltiples conjuntos de datos de entrada, reduciendo significativamente la duplicación de código y mejorando la cobertura de casos límite.
- **Ecosistema de plugins extenso:** Cuenta con más de 1000 plugins disponibles que extienden su funcionalidad para casos específicos como testing de Django, Flask, asyncio, bases de datos, cobertura de código, generación de reportes HTML, ejecución paralela, entre muchos otros.

2.1.2. Ventajas y desventajas

Ventajas [8], [5]:

- **Productividad superior:** La sintaxis minimalista permite escribir tests más rápido, con menos código y mayor claridad, lo que se traduce en mayor cobertura en menos tiempo.
- **Curva de aprendizaje suave:** Los conceptos básicos se dominan rápidamente, permitiendo a desarrolladores nuevos en testing comenzar a escribir tests efectivos en cuestión de minutos.
- **Mensajes de error detallados:** Los fallos se reportan con un nivel de detalle que facilita enormemente la depuración, mostrando exactamente qué valores causaron el fallo y por qué.
- **Fixtures potentes:** El sistema de fixtures es más flexible y potente que los métodos setUp/tearDown tradicionales, permitiendo composición, reutilización y gestión automática de recursos.
- **Comunidad activa:** Cuenta con una comunidad grande y activa que desarrolla constantemente nuevos plugins, proporciona soporte y mantiene documentación actualizada.
- **Compatibilidad hacia atrás:** pytest puede ejecutar tests escritos para unittest sin modificación, facilitando la migración gradual.

Desventajas [8]:

- **Dependencia externa:** Requiere instalación mediante pip, lo que puede ser una consideración en entornos con restricciones de dependencias.
- **Rendimiento en proyectos masivos:** Sin optimización adecuada (como uso de pytest-xdist para paralelización), puede ser más lento que alternativas en proyectos con miles de tests.
- **Complejidad en configuraciones avanzadas:** Aunque simple para casos básicos, configuraciones empresariales complejas pueden requerir conocimiento profundo del sistema de plugins y hooks.

2.1.3. Compatibilidad con sistemas de integración continua (CI/CD)

Pytest ofrece excelente compatibilidad con prácticamente todas las plataformas de CI/CD modernas [6], [11]:

GitHub Actions: Integración directa mediante acciones predefinidas, con soporte para matrices de testing multi-versión de Python y múltiples sistemas operativos.

GitLab CI: Configuración sencilla en `.gitlab-ci.yml`, con capacidad de generar reportes JUnit XML que GitLab puede visualizar nativamente.

Jenkins: Soporte completo mediante el plugin JUnit, con visualización de tendencias históricas y reportes detallados de cobertura.

Travis CI, CircleCI, Azure Pipelines: Configuración estándar con detección automática de tests y generación de reportes.

Generación de reportes: pytest puede generar múltiples formatos de reporte compatibles con CI/CD [10]:

- JUnit XML para visualización en interfaces CI/CD
- Reportes HTML para análisis detallado
- Reportes de cobertura en formato Cobertura XML compatible con SonarQube y CodeCov
- Integración con servicios de análisis de calidad como Codecov, Coveralls y Code Climate

2.1.4. Soporte para la automatización de pruebas

Pytest está diseñado desde su concepción para automatización completa [8]:

Ejecución por línea de comandos: Todas las funcionalidades son accesibles mediante CLI con opciones configurables para selección de tests, niveles de verbosidad, generación de reportes, etc. [10].

Marcadores (markers): Sistema de etiquetado que permite clasificar tests (smoke, integration, slow, etc.) y ejecutar selectivamente subconjuntos específicos mediante `-m` [8].

Plugins de automatización [12]:

- `pytest-watch`: Ejecuta automáticamente tests cuando detecta cambios en archivos
- `pytest-xdist`: Paralelización automática de tests en múltiples CPUs
- `pytest-repeat`: Repite tests múltiples veces para detectar fallos intermitentes
- `pytest-timeout`: Establece timeouts automáticos para prevenir tests colgados

Hooks y plugins personalizados: API completa para crear comportamientos personalizados mediante hooks que se ejecutan en diferentes fases del ciclo de vida del testing [10].

Integración con tox: Compatibilidad nativa con tox para testing automatizado en múltiples versiones de Python y configuraciones de entorno [13].

2.2. Herramienta 2: Unittest

Unittest es el framework de testing incluido en la biblioteca estándar de Python desde la versión 2.1. Basado en JUnit y siguiendo el patrón arquitectónico xUnit, representa el enfoque tradicional y orientado a objetos para pruebas unitarias en Python [1].

2.2.1. Características principales

Unittest implementa un conjunto completo de características siguiendo el modelo xUnit clásico:

- **Arquitectura orientada a objetos:** Los tests se organizan como métodos dentro de clases que heredan de `unittest.TestCase`. Esta estructura proporciona un framework familiar para desarrolladores provenientes de Java o C#.
- **Métodos de configuración y limpieza:** Proporciona métodos especiales `setUp()` y `tearDown()` que se ejecutan antes y después de cada test, así como `setUpClass()` y `tearDownClass()` para inicialización a nivel de clase.
- **Assertions extensivas:** Incluye más de 30 métodos de aserción específicos como `assertEqual()`, `assertTrue()`, `assertRaises()`, `assertIn()`, etc., cada uno optimizado para comparaciones específicas.
- **Test suites y runners:** Permite agrupar tests en suites personalizadas y ejecutarlos mediante runners configurables, ofreciendo control granular sobre la ejecución.
- **Mocking integrado:** Desde Python 3.3, incluye el módulo `unittest.mock` para crear objetos simulados (mocks) y verificar interacciones.
- **Test discovery:** Implementa descubrimiento automático de tests mediante el comando `python -m unittest discover`.

2.2.2. Ventajas y desventajas

Ventajas [14], [3]:

- **Sin dependencias externas:** Al formar parte de la biblioteca estándar, no requiere instalación adicional, lo que simplifica la configuración inicial y el despliegue.
- **Estandarización:** Su estructura basada en xUnit es ampliamente conocida en la industria, facilitando la comprensión del código para desarrolladores con experiencia en otros lenguajes.
- **Documentación oficial:** Cuenta con documentación completa mantenida por el equipo de Python y garantizada estabilidad a largo plazo.
- **Compatibilidad garantizada:** Al ser parte del estándar, la compatibilidad entre versiones de Python está garantizada sin riesgo de dependencias rotas.
- **Ideal para proyectos conservadores:** Perfecto para organizaciones con políticas estrictas sobre dependencias externas o proyectos legacy.

Desventajas [5]:

- **Verbosidad:** Requiere significativamente más código boilerplate comparado con `pytest`, reduciendo la productividad y legibilidad.
- **Herencia obligatoria:** La necesidad de heredar de `TestCase` introduce acoplamiento y limita la flexibilidad en la organización de tests.

- **Mensajes de error básicos:** Los fallos muestran menos contexto que pytest, dificultando la depuración de tests complejos.
- **Fixtures limitadas:** Los métodos setUp/tearDown son menos flexibles que el sistema de fixtures de pytest, especialmente para dependencias complejas.
- **Evolución lenta:** Como parte del estándar de Python, las nuevas características se añaden muy lentamente comparado con proyectos independientes.
- **Ecosistema de extensiones menor:** Cuenta con menos plugins y extensiones disponibles comparado con pytest.

2.2.3. Compatibilidad con sistemas de integración continua (CI/CD)

Unittest mantiene compatibilidad estándar con plataformas CI/CD, aunque generalmente requiere configuración adicional [11]:

Compatibilidad universal: Funciona en cualquier sistema que ejecute Python, con soporte básico en todas las plataformas CI/CD principales [14].

Reportes XML: Requiere bibliotecas adicionales como `xmlrunner` para generar reportes en formato JUnit XML compatible con herramientas de CI/CD [15].

Integración en pipelines [6]:

- GitHub Actions, GitLab CI, Jenkins: Configuración manual mediante comandos `python -m unittest discover`
- Visualización de resultados requiere parseo de salida estándar o uso de extensiones

Limitaciones [5]:

- No incluye generación nativa de reportes HTML
- Menor integración “out of the box” comparado con pytest
- Requiere scripts adicionales para features avanzadas como reportes de cobertura o métricas detalladas

2.2.4. Soporte para la automatización de pruebas

Unittest proporciona capacidades básicas de automatización que pueden extenderse mediante herramientas adicionales [14]:

Ejecución automatizada básica [3]:

- Comando `python -m unittest discover` para descubrimiento automático
- Argumentos de línea de comandos para verbosidad y detención en primer fallo
- Test patterns para filtrado básico de tests

Limitaciones en automatización [5]:

- No incluye sistema de marcadores nativo
- Paralelización requiere herramientas externas
- Menos opciones configurables comparado con pytest

Extensiones disponibles:

- `nose2`: Framework que extiende unittest con características adicionales [16]
- `green`: Runner alternativo con mejor salida y paralelización [17]
- Integración con `tox` para testing multi-entorno [13]

Programación de tests: Puede integrarse con herramientas externas de scheduling como cron, Task Scheduler o sistemas CI/CD para ejecución programada [11].

2.3. Comparación Pytest vs Unittest

La siguiente tabla resume las diferencias clave entre ambas herramientas [8], [5]:

Aspecto	pytest	unittest
Instalación	Requiere <code>pip install pytest</code>	Incluido en Python estándar
Sintaxis	Funciones simples	Clases heredando de <code>TestCase</code>
Boilerplate	Mínimo	Considerable
Fixtures	Sistema avanzado con DI	<code>setUp/tearDown</code> básicos
Assertions	Assert nativo con introspección	Métodos <code>assertEqual</code> , etc.
Mensajes de error	Extremadamente detallados	Básicos
Parametrización	Nativa con decorador	Requiere código manual
Plugins	1000+ disponibles	Ecosistema limitado
Curva aprendizaje	Suave	Moderada
Reportes	Múltiples formatos nativos	Requiere extensiones
CI/CD	Excelente integración	Compatible, setup manual
Paralelización	Plugin <code>pytest-xdist</code>	Herramientas externas
Comunidad	Muy activa	Estable
Casos de uso ideales	Proyectos modernos, startups, desarrollo ágil	Proyectos legacy, restricciones de dependencias

Table 1: Comparación detallada entre pytest y unittest

Para el desarrollo de este proyecto, se ha seleccionado **pytest** como herramienta principal debido a su superior productividad, excelentes mensajes de error y capacidades avanzadas de fixtures que facilitan la creación de tests comprehensivos y mantenibles [8]. Su amplio ecosistema de plugins y mejor integración con herramientas modernas de desarrollo lo convierten en la opción óptima para demostrar las mejores prácticas actuales en testing unitario [5].

3. Configuración y Uso de pytest

Una vez seleccionada la herramienta pytest, es fundamental establecer un entorno de trabajo adecuado que permita aprovechar todas sus capacidades. Este capítulo detalla el proceso completo de instalación, configuración y uso de pytest, desde la configuración inicial hasta la ejecución e interpretación de resultados [8].

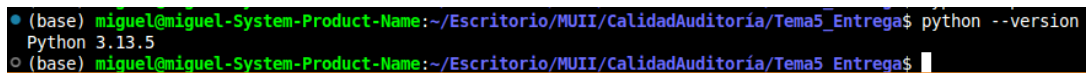
3.1. Instalación de pytest

La instalación de pytest es un proceso sencillo que se realiza mediante el gestor de paquetes pip, incluido por defecto con Python .

3.1.1. Requisitos previos

Antes de instalar pytest, es necesario verificar que Python está correctamente instalado en el sistema. Para ello, se ejecuta el siguiente comando en la terminal:

```
python3 --version
```



```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega$ python --version
Python 3.13.5
○ (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega$

```

Figure 1: Verificación de la versión de python instalada

Este comando debe mostrar la versión de Python instalada (se recomienda Python 3.7 o superior). En nuestro caso, se utilizó Python 3.13.5 .

3.1.2. Proceso de instalación

Para instalar pytest junto con el plugin de cobertura de código, se ejecuta el siguiente comando:

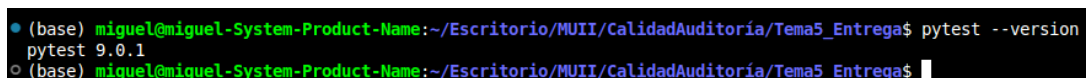
```
pip install pytest pytest-cov
```

Donde:

- pytest: Framework principal de testing
- pytest-cov: Plugin para análisis de cobertura de código

Una vez completada la instalación, se puede verificar que pytest está correctamente instalado comprobando su versión [10]:

```
pytest --version
```



```

• (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega$ pytest --version
pytest 9.0.1
○ (base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega$

```

Figure 2: Verificación de la versión de pytest instalada

La verificación exitosa confirma que pytest está listo para ser utilizado en el desarrollo de pruebas unitarias.

3.2. Configuración del Entorno de Desarrollo

La organización adecuada del proyecto y la configuración de archivos específicos son fundamentales para el correcto funcionamiento de pytest [7].

3.2.1. Estructura del proyecto

Una estructura de proyecto bien organizada facilita la separación entre código fuente y código de pruebas. La estructura recomendada para proyectos con pytest es la siguiente:

```

proyecto_testing/
├── src/
│   ├── __init__.py
│   └── calculadora.py
├── tests/
│   ├── __init__.py
│   └── test_calculadora.py
└── pytest.ini
    
```

Esta estructura presenta varios elementos clave [8]:

- Directorio `src/`: Contiene el código fuente de la aplicación. El archivo `__init__.py` convierte el directorio en un paquete Python, permitiendo importaciones relativas.
- Directorio `tests/`: Almacena todos los archivos de pruebas unitarias. Por convención, los archivos de test comienzan con el prefijo `test_` para que pytest los identifique automáticamente.
- Archivo `pytest.ini`: Contiene la configuración específica de pytest para el proyecto.

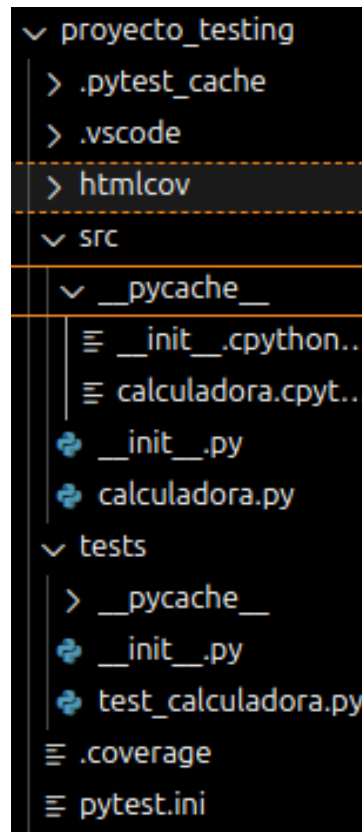


Figure 3: Estructura del proyecto en Visual Studio Code

3.2.2. Archivo de configuración `pytest.ini`

El archivo `pytest.ini` permite personalizar el comportamiento de pytest según las necesidades del proyecto [10]. Este archivo debe ubicarse en la raíz del proyecto y contiene directivas de configuración en formato INI.

La configuración básica recomendada es:

```
[pytest]
testpaths = tests
python_files = test_*.py
python_functions = test_*
addopts = -v --tb=short
```

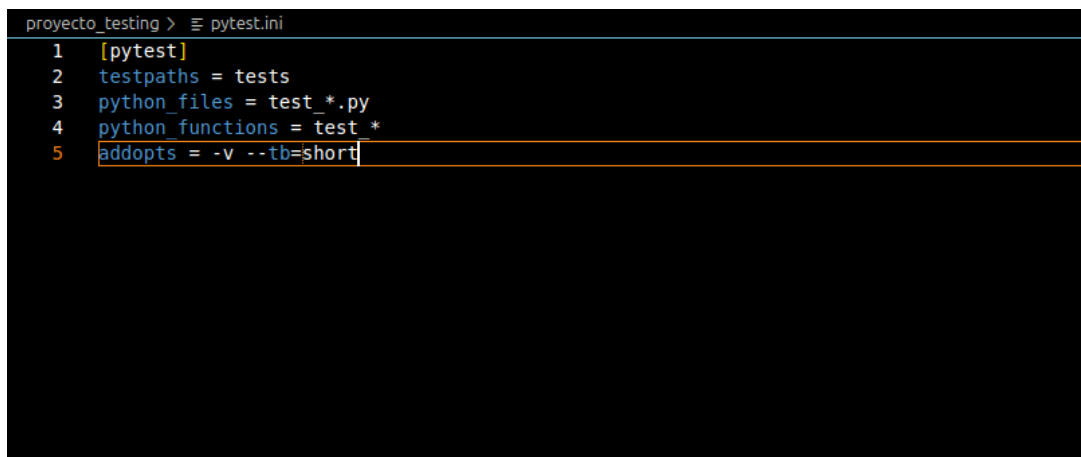
Descripción de cada directiva:

testpaths: Especifica el directorio o directorios donde pytest buscará los archivos de test. Esto acelera el descubrimiento de pruebas al limitar el ámbito de búsqueda.

python_files: Define el patrón de nombres para archivos que contienen tests. El patrón `test_*.py` indica que pytest buscará archivos que comiencen con “test”.

python_functions: Define el patrón de nombres para funciones de test. El patrón `test_*` indica que las funciones de prueba deben comenzar con “test”.

addopts: Opciones adicionales que se aplicarán automáticamente en cada ejecución. La opción `-v` activa el modo verbose (detallado) y `--tb=short` muestra tracebacks abreviados en caso de fallos.



```
proyecto_testing > pytest.ini
1 [pytest]
2 testpaths = tests
3 python_files = test_*.py
4 python_functions = test_*
5 addopts = -v --tb=short
```

Figure 4: Contenido del archivo pytest.ini

Esta configuración garantiza que pytest funcione de manera consistente y proporcione información detallada sobre la ejecución de las pruebas [8].

3.2.3. Integración con entornos de desarrollo

Aunque pytest puede ejecutarse completamente desde la línea de comandos, los entornos de desarrollo integrados (IDEs) modernos como Visual Studio Code o PyCharm ofrecen integración nativa con pytest. Estos IDEs permiten:

- Visualización gráfica de la estructura de tests
- Ejecución individual de tests con un clic
- Depuración de tests con breakpoints
- Visualización de resultados en tiempo real

La configuración de estos entornos es opcional pero recomendada para mejorar la productividad durante el desarrollo [5].

3.3. Creación de Pruebas Unitarias

Las pruebas unitarias en pytest se crean utilizando funciones Python simples decoradas y organizadas según convenciones específicas [8]. A continuación se explican los conceptos fundamentales para crear pruebas efectivas.

3.3.1. Estructura básica de un test

Un test en pytest es una función Python que comienza con el prefijo `test_` y utiliza aserciones para verificar el comportamiento esperado [10]:

```
def test_nombre_descriptivo():
    # Preparación (Arrange)
    valor = funcion_a_probar()

    # Verificación (Assert)
    assert valor == resultado_esperado
```

Esta estructura sigue el patrón AAA (Arrange-Act-Assert):

- **Arrange:** Preparar los datos y el contexto necesario
- **Act:** Ejecutar la funcionalidad a probar
- **Assert:** Verificar que el resultado es el esperado

3.3.2. Fixtures: Reutilización de código de prueba

Las fixtures son funciones especiales que proporcionan datos o recursos reutilizables para múltiples tests [8]. Se definen mediante el decorador `@pytest.fixture`:

```
@pytest.fixture
def recurso_compartido():
    """Proporciona un recurso para los tests"""
    return objeto_a_compartir
```

Las fixtures ofrecen varias ventajas:

- Reducen la duplicación de código
- Gestionan automáticamente la configuración y limpieza de recursos
- Permiten la composición mediante la solicitud de otras fixtures
- Soportan diferentes alcances (function, class, module, session)

Para utilizar una fixture, simplemente se incluye como parámetro en la función de test [10]:

```
def test_usando_fixture(recurso_compartido):
    assert recurso_compartido.metodo() == valor_esperado
```

pytest inyecta automáticamente la fixture cuando ejecuta el test, proporcionando el recurso necesario sin necesidad de código adicional.

3.3.3. Aserciones: Verificación de resultados

pytest utiliza la sentencia `assert` nativa de Python para verificar condiciones [10]. Cuando una aserción falla, pytest proporciona información detallada sobre el fallo mediante introspección del código:

```
def test_suma():
    resultado = 2 + 2
    assert resultado == 4 # Pasa

def test_suma_fallido():
```

```
resultado = 2 + 2
assert resultado == 5 # Falla con información detallada
```

Las aserciones pueden verificar:

- Igualdad y desigualdad
- Valores booleanos
- Contenido en colecciones
- Tipos de datos
- Cualquier expresión Python que retorne True/False

La ventaja de usar `assert` en lugar de métodos específicos es la simplicidad y la información detallada que `pytest` proporciona al fallar [8].

3.3.4. Pruebas parametrizadas

La parametrización permite ejecutar el mismo test con múltiples conjuntos de datos de entrada, evitando la duplicación de código [10]. Se implementa mediante el decorador `@pytest.mark.parametrize`:

```
@pytest.mark.parametrize("entrada,esperado", [
    (1, 2),
    (2, 4),
    (3, 6),
])
def test_duplicar(entrada, esperado):
    assert entrada * 2 == esperado
```

Este test se ejecutará tres veces, una por cada conjunto de parámetros. La parametrización es especialmente útil para:

- Probar casos límite
- Verificar múltiples escenarios con la misma lógica
- Mantener el código de test conciso y legible

`pytest` muestra cada caso parametrizado individualmente en los resultados, facilitando la identificación de casos específicos que fallan [8].

3.3.5. Verificación de excepciones

Para verificar que el código lanza las excepciones esperadas en situaciones de error, `pytest` proporciona el contexto `pytest.raises()` [10]:

```
def test_division_por_cero():
    with pytest.raises(ZeroDivisionError):
        resultado = 10 / 0
```

Este contexto permite:

- Verificar el tipo de excepción lanzada
- Verificar el mensaje de la excepción mediante expresiones regulares
- Asegurar que el código maneja correctamente situaciones excepcionales

Opcionalmente, se puede verificar el mensaje de la excepción:

```
def test_excepcion_con_mensaje():
    with pytest.raises(ValueError, match="mensaje esperado"):
        raise ValueError("mensaje esperado")
```

La verificación de excepciones es fundamental para probar el manejo robusto de errores en el código [9].

3.4. Ejecución de Pruebas

pytest ofrece múltiples formas de ejecutar las pruebas, proporcionando diferentes niveles de detalle y opciones de filtrado [10].

3.4.1. Ejecución básica

Para ejecutar todos los tests del proyecto, se utiliza el comando más simple:

`pytest`

Este comando:

- Busca automáticamente todos los archivos que coincidan con `test_*.py`
- Ejecuta todas las funciones que comiencen con `test_`
- Muestra un resumen de los resultados

```
5_Entrega/proyecto_testing$ pytest
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.0.1, pluggy-1.5.0 -- /home/miguel/miniconda3/bin/python3.13
cachedir: .pytest_cache
metadata: {'Python': '3.13.5', 'Platform': 'Linux-6.14.0-36-generic-x86_64-with-glibc2.39', 'Packages': {'py': '3.11.1', 'html': '4.1.1'}}
rootdir: /home/miguel/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega/proyecto_testing
configfile: pytest.ini
testpaths: tests
plugins: cov-7.0.0, metadata-3.1.1, html-4.1.1
collected 11 items

tests/test_calculadora.py::test_sumar_positivos PASSED [ 9%]
tests/test_calculadora.py::test_sumar_negativos PASSED [ 18%]
tests/test_calculadora.py::test_restar PASSED [ 27%]
tests/test_calculadora.py::test_multiplicar PASSED [ 36%]
tests/test_calculadora.py::test_multiplicar_por_cero PASSED [ 45%]
tests/test_calculadora.py::test_dividir PASSED [ 54%]
tests/test_calculadora.py::test_dividir_por_cero PASSED [ 63%]
tests/test_calculadora.py::test_sumar_parametrizado[2-3-5] PASSED [ 72%]
tests/test_calculadora.py::test_sumar_parametrizado[0-0-0] PASSED [ 81%]
tests/test_calculadora.py::test_sumar_parametrizado[-1-1-0] PASSED [ 90%]
tests/test_calculadora.py::test_sumar_parametrizado[100-200-300] PASSED [100%]

===== 11 passed in 0.01s =====
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega/proyecto_testing$
```

Figure 5: Ejecución básica de pytest

La salida básica muestra:

- Número de tests recolectados
- Progreso de ejecución con puntos o letras (. para PASSED, F para FAILED)
- Resumen final con estadísticas

3.4.2. Ejecución en modo verbose

Para obtener información detallada sobre cada test ejecutado, se utiliza la opción `-v` (verbose) [10]:

`pytest -v`

El modo verbose proporciona:

- Nombre completo de cada test (módulo::clase::función)
- Estado individual de cada test (PASSED, FAILED, SKIPPED)
- Porcentaje de progreso
- Información de plugins activos

```
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega/proyecto_testing$ pytest -v
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.0.1, pluggy-1.5.0 -- /home/miguel/miccondas3/bin/python3.13
cachedir: .pytest_cache
metadata: {'Python': '3.13.5', 'Platform': 'Linux-6.14.0-36-generic-x86_64-with-glibc2.39', 'Packages': {'pytest': '9.0.1', 'pluggy': '1.5.0'}, 'Plugins': {'cov': '7.0.0', 'metadata': '3.1.1', 'html': '4.1.1'}}
rootdir: /home/miguel/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega/proyecto_testing
configfile: pytest.ini
testpaths: tests
plugins: cov-7.0.0, metadata-3.1.1, html-4.1.1
collected 11 items

tests/test_calculadora.py::test_sumar_positivos PASSED [ 9%]
tests/test_calculadora.py::test_sumar_negativos PASSED [ 18%]
tests/test_calculadora.py::test_restar PASSED [ 27%]
tests/test_calculadora.py::test_multiplicar PASSED [ 36%]
tests/test_calculadora.py::test_multiplicar_por_cero PASSED [ 45%]
tests/test_calculadora.py::test_dividir PASSED [ 54%]
tests/test_calculadora.py::test_dividir_por_cero PASSED [ 63%]
tests/test_calculadora.py::test_sumar_parametrizado[2-3-5] PASSED [ 72%]
tests/test_calculadora.py::test_sumar_parametrizado[0-0-0] PASSED [ 81%]
tests/test_calculadora.py::test_sumar_parametrizado[-1-1-0] PASSED [ 90%]
tests/test_calculadora.py::test_sumar_parametrizado[100-200-300] PASSED [100%]

===== 11 passed in 0.01s =====
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega/proyecto_testing$
```

Figure 6: Ejecución en modo verbose mostrando cada test individualmente

Este modo es especialmente útil durante el desarrollo para identificar rápidamente qué tests específicos están fallando [8].

3.4.3. Análisis de cobertura de código

La cobertura de código mide qué porcentaje del código fuente es ejecutado durante las pruebas [18]. Para generar un reporte de cobertura, se utiliza:

```
pytest --cov=src --cov-report=term
```

Donde:

- `--cov=src`: Indica el directorio o paquete a analizar
- `--cov-report=term`: Genera el reporte en la terminal

```
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega/proyecto_testing$ pytest --cov=src --cov-report=term
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.0.1, pluggy-1.5.0 -- /home/miguel/miccondas3/bin/python3.13
cachedir: .pytest_cache
metadata: {'Python': '3.13.5', 'Platform': 'Linux-6.14.0-36-generic-x86_64-with-glibc2.39', 'Packages': {'pytest': '9.0.1', 'pluggy': '1.5.0'}, 'Plugins': {'cov': '7.0.0', 'metadata': '3.1.1', 'html': '4.1.1'}}
rootdir: /home/miguel/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega/proyecto_testing
configfile: pytest.ini
testpaths: tests
plugins: cov-7.0.0, metadata-3.1.1, html-4.1.1
collected 11 items

tests/test_calculadora.py::test_sumar_positivos PASSED [ 9%]
tests/test_calculadora.py::test_sumar_negativos PASSED [ 18%]
tests/test_calculadora.py::test_restar PASSED [ 27%]
tests/test_calculadora.py::test_multiplicar PASSED [ 36%]
tests/test_calculadora.py::test_multiplicar_por_cero PASSED [ 45%]
tests/test_calculadora.py::test_dividir PASSED [ 54%]
tests/test_calculadora.py::test_dividir_por_cero PASSED [ 63%]
tests/test_calculadora.py::test_sumar_parametrizado[2-3-5] PASSED [ 72%]
tests/test_calculadora.py::test_sumar_parametrizado[0-0-0] PASSED [ 81%]
tests/test_calculadora.py::test_sumar_parametrizado[-1-1-0] PASSED [ 90%]
tests/test_calculadora.py::test_sumar_parametrizado[100-200-300] PASSED [100%]

===== tests coverage =====
coverage: platform linux, python 3.13.5-final-0

Name                Stmts   Miss  Cover
-----
src/__init__.py         0      0   100%
src/calculadora.py     11      0   100%
TOTAL                  11      0   100%

===== 11 passed in 0.02s =====
(base) miguel@miguel-System-Product-Name:~/Escritorio/MUII/CalidadAuditoria/Tema5_Entrega/proyecto_testing$
```

Figure 7: Reporte de cobertura mostrando el porcentaje de código probado

El reporte de cobertura muestra:

- **Name**: Archivo analizado
- **Stmts**: Número de líneas ejecutables
- **Miss**: Líneas no ejecutadas durante los tests
- **Cover**: Porcentaje de cobertura

Una cobertura alta (cercana al 100%) indica que la mayoría del código está siendo probado, aunque no garantiza la ausencia de bugs [9]. Es importante combinar alta cobertura con tests de calidad que verifiquen comportamientos específicos.

3.4.4. Interpretación de resultados

La salida de pytest proporciona información estructurada sobre la ejecución de las pruebas [10]. Los elementos clave a interpretar son:

Cabecera de sesión:

```
===== test session starts =====  
platform linux -- Python 3.13.5, pytest-9.0.1, pluggy-1.5.0
```

Muestra el sistema operativo, versión de Python y versión de pytest utilizadas.

Recolección de tests:

```
collected 11 items
```

Indica cuántos tests fueron encontrados y se ejecutarán.

Resultados individuales:

```
tests/test_calculadora.py::test_sumar_positivos PASSED [ 9%]
```

Cada línea muestra el archivo, nombre del test, resultado y progreso.

Estados posibles:

- PASSED: El test se ejecutó correctamente
- FAILED: El test falló (se muestra información detallada del error)
- SKIPPED: El test fue omitido
- ERROR: Error durante la configuración del test

Resumen final:

```
===== 11 passed in 0.01s =====
```

Muestra el total de tests por estado y el tiempo de ejecución total.

Cuando un test falla, pytest proporciona información detallada incluyendo:

- La línea exacta donde ocurrió el fallo
- Los valores de las variables involucradas
- El traceback completo del error
- Diferencias entre valores esperados y obtenidos

Esta información detallada facilita enormemente la depuración y resolución de problemas [8].

4. Desarrollo Práctico de Pruebas Unitarias

Este capítulo presenta la implementación práctica de un sistema software completo con su correspondiente suite de pruebas unitarias, demostrando la aplicación real de los conceptos y técnicas previamente descritos [5].

4.1. Sistema Desarrollado: Calculadora

Para demostrar el proceso completo de desarrollo guiado por pruebas, se ha implementado una clase Calculadora que proporciona operaciones aritméticas básicas [8].

4.1.1. Descripción del componente

La calculadora implementa cuatro operaciones fundamentales:

- **Suma:** Adición de dos números
- **Resta:** Sustracción de dos números
- **Multiplicación:** Producto de dos números
- **División:** Cociente de dos números con validación de división por cero

Este sistema, aunque simple, proporciona suficiente complejidad para demostrar diferentes tipos de pruebas unitarias, incluyendo casos normales, casos límite y manejo de excepciones [9].

4.1.2. Implementación del código fuente

El código fuente se encuentra en el archivo `src/calculadora.py`:

```
class Calculadora:
    """Calculadora simple para operaciones básicas"""

    def sumar(self, a, b):
        """Suma dos números"""
        return a + b

    def restar(self, a, b):
        """Resta dos números"""
        return a - b

    def multiplicar(self, a, b):
        """Multiplica dos números"""
        return a * b

    def dividir(self, a, b):
        """Divide dos números"""
        if b == 0:
            raise ValueError("No se puede dividir por cero")
        return a / b
```

```

proyecto_testing > src > calculadora.py > ...

1  class Calculadora:
2      """Calculadora simple para operaciones básicas"""
3
4      def sumar(self, a, b):
5          """Suma dos números"""
6          return a + b
7
8      def restar(self, a, b):
9          """Resta dos números"""
10         return a - b
11
12         def multiplicar(self, a, b):
13             """Multiplica dos números"""
14             return a * b
15
16         def dividir(self, a, b):
17             """Divide dos números"""
18             if b == 0:
19                 raise ValueError("No se puede dividir por cero")
20             return a / b

```

Figure 8: Implementación de la clase Calculadora

4.1.3. Características de implementación

El código presenta varias características importantes desde el punto de vista de testing [7]:

Métodos independientes: Cada operación está implementada en un método separado, facilitando el testing aislado de cada funcionalidad.

Validación de entrada: El método `dividir()` valida que el divisor no sea cero, lanzando una excepción `ValueError` con un mensaje descriptivo. Esta validación permite probar el manejo correcto de errores.

Documentación clara: Cada método incluye un docstring que describe su funcionalidad, mejorando la legibilidad y el mantenimiento del código.

Simplicidad: La implementación es directa y sin efectos secundarios, lo que facilita la creación de pruebas predecibles y repetibles.

4.2. Pruebas Implementadas

La suite de pruebas unitarias se encuentra en el archivo `tests/test_calculadora.py` y cubre exhaustivamente todas las funcionalidades de la calculadora [8].

4.2.1. Código de las pruebas unitarias

```
import pytest
from src.calculadora import Calculadora

# Fixture para crear una instancia de calculadora
@pytest.fixture
def calc():
    """Proporciona una instancia de Calculadora para los tests"""
    return Calculadora()

# Tests de suma
def test_sumar_positivos(calc):
    """Test: Sumar dos números positivos"""
    resultado = calc.sumar(3, 5)
    assert resultado == 8

def test_sumar_negativos(calc):
    """Test: Sumar dos números negativos"""
    resultado = calc.sumar(-3, -5)
    assert resultado == -8

# Tests de resta
def test_restar(calc):
    """Test: Restar dos números"""
    resultado = calc.restar(10, 3)
    assert resultado == 7

# Tests de multiplicación
def test_multiplicar(calc):
    """Test: Multiplicar dos números"""
    resultado = calc.multiplicar(4, 5)
    assert resultado == 20

def test_multiplicar_por_cero(calc):
    """Test: Multiplicar por cero"""
    resultado = calc.multiplicar(5, 0)
    assert resultado == 0

# Tests de división
def test_dividir(calc):
    """Test: Dividir dos números"""
    resultado = calc.dividir(10, 2)
    assert resultado == 5

def test_dividir_por_cero(calc):
    """Test: Dividir por cero debe lanzar excepción"""
    with pytest.raises(ValueError, match="No se puede dividir por cero"):
        calc.dividir(10, 0)

# Test parametrizado
@pytest.mark.parametrize("a,b,esperado", [
```

```
(2, 3, 5),
(0, 0, 0),
(-1, 1, 0),
(100, 200, 300),
])
def test_sumar_parametrizado(calc, a, b, esperado):
    """Test parametrizado: Múltiples casos de suma"""
    assert calc.sumar(a, b) == esperado
```

```

proyecto_testing > tests > test_calculadora.py > ...

1  import pytest
2  from src.calculadora import Calculadora
3
4  # Fixture para crear una instancia de calculadora
5  @pytest.fixture
6  def calc():
7      """Proporciona una instancia de Calculadora para los tests"""
8      return Calculadora()
9
10 # Tests de suma
11 def test_sumar_positivos(calc):
12     """Test: Sumar dos números positivos"""
13     resultado = calc.sumar(3, 5)
14     assert resultado == 8
15
16 def test_sumar_negativos(calc):
17     """Test: Sumar dos números negativos"""
18     resultado = calc.sumar(-3, -5)
19     assert resultado == -8
20
21 # Tests de resta
22 def test_restar(calc):
23     """Test: Restar dos números"""
24     resultado = calc.restar(10, 3)
25     assert resultado == 7
26
27 # Tests de multiplicación
28 def test_multiplicar(calc):
29     """Test: Multiplicar dos números"""
30     resultado = calc.multiplicar(4, 5)
31     assert resultado == 20
32
33 def test_multiplicar_por_cero(calc):
34     """Test: Multiplicar por cero"""
35     resultado = calc.multiplicar(5, 0)
36     assert resultado == 0
37
38 # Tests de división
39 def test_dividir(calc):
40     """Test: Dividir dos números"""
41     resultado = calc.dividir(10, 2)
42     assert resultado == 5
43
44 def test_dividir_por_cero(calc):
45     """Test: Dividir por cero debe lanzar excepción"""
46     with pytest.raises(ValueError, match="No se puede dividir por cero"):
47         calc.dividir(10, 0)
48
49 # Test parametrizado
50 @pytest.mark.parametrize("a,b,esperado", [
51     (2, 3, 5),
52     (0, 0, 0),
53     (-1, 1, 0),
54     (100, 200, 300),
55 ])
56 def test_sumar_parametrizado(calc, a, b, esperado):
57     """Test parametrizado: Múltiples casos de suma"""
58     assert calc.sumar(a, b) == esperado

```

Figure 9: Implementación completa de las pruebas unitarias

4.2.2. Análisis de las pruebas implementadas

La suite de pruebas implementa 11 tests que cubren diferentes aspectos del sistema [9]:

Fixture compartida: La función `calc()` proporciona una instancia limpia de Calculadora a cada test, garantizando independencia entre pruebas y evitando efectos secundarios.

Tests de suma (3 tests):

- `test_sumar_positivos`: Verifica la suma de números positivos
- `test_sumar_negativos`: Prueba con números negativos
- `test_sumar_parametrizado`: Ejecuta 4 casos diferentes mediante parametrización (0+0, valores positivos, negativos, números grandes)

Tests de resta (1 test):

- `test_restar`: Verifica la operación de resta básica

Tests de multiplicación (2 tests):

- `test_multiplicar`: Prueba la multiplicación estándar
- `test_multiplicar_por_cero`: Caso límite importante de multiplicación por cero

Tests de división (2 tests):

- `test_dividir`: Verifica la división normal
- `test_dividir_por_cero`: Prueba el manejo de excepción para división por cero

4.2.3. Justificación de casos de prueba

La selección de casos de prueba sigue principios establecidos de testing de software [19]:

Casos normales: Se prueban operaciones típicas con valores comunes para verificar el funcionamiento básico.

Casos límite: Se incluyen tests con cero, números negativos y números grandes para identificar comportamientos en los límites del dominio.

Casos de error: Se verifica explícitamente el manejo de situaciones excepcionales como la división por cero.

Diversidad de entrada: La parametrización permite probar múltiples combinaciones de entrada con el mismo código de test, maximizando la cobertura con mínimo esfuerzo.

Esta estrategia de testing proporciona confianza razonable en el correcto funcionamiento del sistema dentro de su dominio de aplicación [8].

4.3. Resultados Obtenidos

La ejecución de la suite completa de pruebas proporciona información valiosa sobre la calidad y corrección del sistema implementado [9].

4.3.1. Ejecución exitosa de las pruebas

Al ejecutar `pytest -v`, se obtiene el siguiente resultado:

```
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.0.1, pluggy-1.5.0
cachedir: .pytest_cache
rootdir: /home/miguel/Escritorio/.../proyecto_testing
configfile: pytest.ini
testpaths: tests
plugins: cov-7.0.0, metadata-3.1.1, html-4.1.1
collected 11 items

tests/test_calculadora.py::test_sumar_positivos PASSED [ 9%]
tests/test_calculadora.py::test_sumar_negativos PASSED [ 18%]
tests/test_calculadora.py::test_restar PASSED [ 27%]
tests/test_calculadora.py::test_multiplicar PASSED [ 36%]
tests/test_calculadora.py::test_multiplicar_por_cero PASSED [ 45%]
tests/test_calculadora.py::test_dividir PASSED [ 54%]
tests/test_calculadora.py::test_dividir_por_cero PASSED [ 63%]
tests/test_calculadora.py::test_sumar_parametrizado[2-3-5] PASSED [ 72%]
tests/test_calculadora.py::test_sumar_parametrizado[0-0-0] PASSED [ 81%]
tests/test_calculadora.py::test_sumar_parametrizado[-1-1-0] PASSED [ 90%]
tests/test_calculadora.py::test_sumar_parametrizado[100-200-300] PASSED [100%]

===== 11 passed in 0.01s =====
```

Análisis de resultados:

Tasa de éxito: 11 de 11 tests pasaron exitosamente (100%), indicando que todas las funcionalidades implementadas cumplen con las especificaciones esperadas.

Tiempo de ejecución: 0.01 segundos, demostrando que la suite de pruebas es extremadamente eficiente y puede ejecutarse frecuentemente sin impactar la productividad.

Tests parametrizados: Los 4 casos parametrizados se muestran individualmente, facilitando la identificación de casos específicos en caso de fallos futuros.

4.3.2. Análisis de cobertura de código

El análisis de cobertura proporciona métricas cuantitativas sobre qué porcentaje del código está siendo ejercitado por las pruebas [18]:

```
===== tests coverage =====
coverage: platform linux, python 3.13.5-final-0

Name                Stmts   Miss  Cover
-----
src/__init__.py         0      0  100%
src/calculadora.py     11      0  100%
-----
TOTAL                  11      0  100%
```

Interpretación del reporte:

Cobertura total: 100% (11 de 11 líneas ejecutables fueron ejecutadas)

Líneas no ejecutadas: 0 (Miss = 0), indicando que todos los caminos de código fueron probados

Cobertura por archivo:

- `__init__.py`: 100% (archivo vacío)
- `calculadora.py`: 100% (todas las líneas de código fueron ejecutadas)

Una cobertura del 100% indica que:

- Todos los métodos fueron llamados
- Todas las ramas condicionales fueron evaluadas (if/else en `dividir`)
- Todas las rutas de ejecución fueron ejercitadas
- Tanto los casos de éxito como los casos de error fueron probados

4.3.3. Interpretación de calidad del sistema

Los resultados obtenidos permiten evaluar la calidad del sistema desarrollado según varios criterios [19]:

Corrección funcional: Todos los tests pasaron, confirmando que el sistema implementa correctamente las operaciones aritméticas especificadas.

Robustez: El test de división por cero confirma que el sistema maneja correctamente situaciones excepcionales, lanzando excepciones apropiadas en lugar de fallar silenciosamente.

Cobertura completa: El 100% de cobertura garantiza que no hay código sin probar, reduciendo el riesgo de bugs ocultos en código no ejercitado.

Mantenibilidad: La suite de pruebas automatizada facilita futuras modificaciones del código, ya que cualquier regresión será detectada inmediatamente al ejecutar los tests.

Confiabilidad: La ejecución rápida y consistente de los tests proporciona confianza en que el sistema funciona correctamente en diferentes escenarios.

4.3.4. Limitaciones y trabajo futuro

Aunque los resultados son satisfactorios, es importante reconocer algunas limitaciones [9]:

Cobertura no garantiza ausencia de bugs: El 100% de cobertura significa que todas las líneas fueron ejecutadas, pero no garantiza que se probaron todas las combinaciones posibles de entrada ni todos los escenarios de uso.

Tests limitados a casos conocidos: Los tests verifican comportamientos esperados basados en la especificación, pero podrían no detectar requisitos no documentados o casos de uso no anticipados.

Ausencia de tests de integración: Las pruebas unitarias verifican componentes aislados; tests de integración adicionales serían necesarios para verificar la interacción con otros componentes en un sistema más grande.

Para mejorar la calidad del testing en iteraciones futuras, se podrían considerar:

- Tests de propiedades con generación automática de casos de prueba
- Tests de rendimiento para operaciones con números muy grandes
- Tests de concurrencia si el sistema se utiliza en contextos multi-hilo
- Análisis de mutación para verificar la efectividad de los tests existentes

No obstante, para el alcance del sistema actual, la suite de pruebas implementada proporciona un nivel de confianza adecuado en su corrección y robustez [8].

Bibliografía

- [1] K. Beck, *Test Driven Development: By Example*. Boston, MA: Addison-Wesley, 2002.
- [2] I. Sommerville, *Software Engineering*, 10th ed. Harlow, England: Pearson, 2015.
- [3] M. Lutz, *Learning Python*, 5th ed. Sebastopol, CA: O'Reilly Media, 2013.
- [4] L. Ramalho, *Fluent Python: Clear, Concise, and Effective Programming*. Sebastopol, CA: O'Reilly Media, 2015.
- [5] H. J. Percival, *Test-Driven Development with Python*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2017.
- [6] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley, 2010.
- [7] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall, 2008.
- [8] B. Okken, *Python Testing with pytest: Simple, Rapid, Effective, and Scalable*, 2nd ed. Raleigh, NC: The Pragmatic Bookshelf, 2022.
- [9] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ: John Wiley & Sons, 2011.
- [10] pytest Development Team, “pytest Documentation.” Accessed: Dec. 04, 2024. [Online]. Available: <https://docs.pytest.org/>
- [11] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Boston, MA: Addison-Wesley, 2015.
- [12] pytest Development Team, “pytest Plugin List.” Accessed: Dec. 04, 2024. [Online]. Available: https://docs.pytest.org/en/latest/reference/plugin_list.html
- [13] tox Development Team, “tox Documentation.” Accessed: Dec. 04, 2024. [Online]. Available: <https://tox.wiki/>
- [14] Python Software Foundation, “unittest — Unit testing framework.” Accessed: Dec. 04, 2024. [Online]. Available: <https://docs.python.org/3/library/unittest.html>
- [15] J. F. Smart, *Jenkins: The Definitive Guide*. Sebastopol, CA: O'Reilly Media, 2011.
- [16] nose2 Development Team, “nose2 Documentation.” Accessed: Dec. 04, 2024. [Online]. Available: <https://docs.nose2.io/>
- [17] Green Development Team, “Green Test Runner.” Accessed: Dec. 04, 2024. [Online]. Available: <https://github.com/CleanCut/green>
- [18] Coverage.py Development Team, “Coverage.py Documentation.” Accessed: Dec. 04, 2024. [Online]. Available: <https://coverage.readthedocs.io/>
- [19] D. Graham, E. van Veenendaal, I. Evans, and R. Black, *Foundations of Software Testing: ISTQB Certification*. London, UK: Cengage Learning, 2008.